

Data Cache Prefetching With Dynamic Adaptation

MINHAJ AHMAD KHAN*

Department of Computer Science, Bahauddin Zakariya University, Multan, Pakistan

**Corresponding author: minhajahmad@gmail.com*

Modern processors based on VLIW architecture rely heavily on software cache prefetching incorporated by the compiler. For accurate prefetching different factors such as latencies of the loop iterations need to be taken into account, which cannot be determined at (static) compile time. Consequently, the compilers either produce inaccurate prefetches or resort to producing code without prefetching. Many applications with complex code are therefore unable to perform very well on the modern processors. In this paper, we present an approach that is able to generate accurate prefetch instructions by exploiting information available at runtime. The code is instrumented with prefetches having offsets which may be adapted at runtime through a dynamic code specializer. Such cache prefetching with dynamic adaptation results in better performance of the applications. The runtime code generation activity is highly efficient and incurs a very small overhead. The experiments performed on Itanium-II architecture using *icc* and *gcc* compilers produce an average speedup of 2.51% and 2.56%, respectively.

Keywords: code optimization; cache prefetching; dynamic compilation; programming languages; memory hierarchy optimization

Received 7 June 2009; revised 2 January 2010

Handling editor: Simon Moore

1. INTRODUCTION

To bridge the latency gap between memory and the processor, modern computer architectures incorporate multiple levels of memory hierarchy. This gap exists due to rapidly increasing speed of microprocessors in contrast with the speed of memory architectures. During execution of an application, the data required for computation needs to be brought into the processor cache from the main memory. The availability of data in the processor cache allows the processor to obtain peak performance for the application being executed.

Most of the applications suffer from performance degradation due to large latencies of memory access. In this regard, software cache prefetching is the technique that causes data to be fetched from memory into the cache before it is actually used by the processor. In the meanwhile, other instructions can be scheduled to execute, thereby mitigating the memory delay that could otherwise incur due to unavailability of data in the processor cache. This mechanism results in better performance of the applications as the cache miss penalties do not occur if the processor finds the required data in the cache.

The cache prefetching needs calculation of addresses of data required by the processor together with the prefetch distance

which refers to the address of the next element that must be brought into the cache. The prefetch distance mainly depends upon the latency of cache miss and the latency of the loop iteration, i.e. the execution time taken by an iteration of the loop [1–4]. The latency of the loop iteration cannot be determined (statically) at compile time and, therefore, it results in inaccurate prefetching of data.

This paper suggests a novel strategy that aims at improving the cache prefetching through dynamic adaptation. The latencies are calculated dynamically and the necessary code is modified at runtime to produce accurate prefetch instructions. To accomplish this, the code is first instrumented with prefetch instructions containing offsets. The compiler generates the code with some instructions that are dependent on these offsets. These instructions can be traced at compile time and modified at runtime to adjust the prefetch distance in accordance with the latencies of the loop iterations.

The modification of code is performed by a runtime specializer making use of information collected at compile time. The specializer inserts new values for a few instructions at precomputed locations of the binary code. Since the specializer modifies a small number of binary instructions at runtime,

the approach incurs a very small overhead. In addition, all the analyses required for adaptation are performed at compile time. The reduced runtime overhead and compile time analyses result in effective cache prefetching and thereby improve the performance of the applications.

The rest of the paper is organized as follows. Section 2 describes the context of the dynamic cache prefetching strategy. The steps necessary to implement and automate the approach of runtime data cache prefetching have been elaborated in Section 3. Section 4 describes the architecture of the implementation framework. The experimental setup and results have been presented in Section 5. Section 6 describes the existing research work related to cache prefetching followed by a conclusion and the future work given in Section 7.

2. CONTEXT OF DYNAMIC CACHE PREFETCHING STRATEGY

Dynamic cache prefetching is used to bring data into the cache at runtime, i.e. during the execution of the application. The data prefetching requires the prefetch distance to be computed which specifies the address of the next element to be brought into the cache. The distance depends upon the latency of the loop code. Unfortunately, the prefetching distance may only be computed correctly when the latencies of the loop execution are known. Due to the large runtime overhead of data cache prefetching, most of the dynamic prefetching strategies [2–5] resort to estimating the distance just like static prefetching strategies, which is not always beneficial. The dynamic cache prefetching therefore requires adaptation of distance in accordance with the runtime behavior.

For instance, consider the code snippet given in Fig. 1. The latency of a single iteration varies depending upon the condition given in the loop. The cache prefetching based on the estimates actually deteriorates the performance due to unpredictable behavior of the code. This issue can be addressed dynamically by computing the exact distance for frequently missing loads.

An adapter distance may be initially inserted (statically) at compile time. For example, for IA-64 architecture, the instruction of *lfetch* ($&X[i + \text{offset}]$) may be used to prefetch data of some variable existing at the distance as computed in the offset.

If the code is specialized with different values of offsets,¹ we find that the object code versions obtained after compilation are very much similar. For example, if the same code is specialized with the offset values of 5 and 7, we obtain two versions of code as given in Fig. 2a and b, respectively. These versions have similar code scheduling, register allocation and optimizations applied to them. The immediate values² in the object code

```
#define size 1000
double function(double * X, double * Y, double alpha)
{
    double Sum = 0.0;
    for (int i=0; i < size; i++)
        if (X[i] == Y[i])
            Sum = Sum + X[i];
        ...
    else
        X[i] += i * X[i];
        Y[i] = Y[i] + alpha * X[i];
        Sum = Sum + Y[i];
        ...
    }
    return Sum;
}
```

FIGURE 1. Code snippet with varying latency of its work in a single iteration.

<pre>add r15=5, r10 lfetch.nt1 [r15] add r17=736, r11 ;; lfetch.nt1 [r17]</pre>	<pre>add r15=7, r10 lfetch.nt1 [r15] add r17=768, r11 ;; lfetch.nt1 [r17]</pre>
(a)	(b)

FIGURE 2. Assembly code generated by icc v 8.0 on IA-64 after instrumentation with *lfetch* instructions: (a) code with offset = 5 and (b) code with offset = 7.

can be regenerated for new distances found at runtime. The modification of code may then result from the adjustment of the prefetch distance at runtime. This is accomplished by using a single version termed as *template* and modifying the object code instructions through a very efficient code termed as *template specializer*, which may adapt dynamically the prefetch distance existing in the *template*. Since the *template* generation, *specializer* generation and computation of the locations to be modified are entirely performed at (static) compile time, the runtime overhead is very small.

The prefetching with dynamic adaptation requiring modification of a small number of instructions therefore results in better performance. This approach is in contrast to other existing dynamic prefetching systems [3, 4] which make use of one or more threads, and generate entirely new code at runtime.

3. APPROACH OF RUNTIME DATA CACHE PREFETCHING

The static information available to a compiler is not sufficient enough to incorporate accurate prefetches in the code. The compiler decisions are based on heuristics exploiting only static information. The fact that the behavior of an application differs in accordance with different input, may make the application performance to degrade. This limitation cannot be overcome

¹Integer values generate very similar versions of code after compilation. These versions are required for the subsequent dynamic adaptation of code.

²The number of such immediate values depends upon the instances of the offset, unroll factor and the loop versioning.

by static compilation. Therefore, we address this problem by performing runtime data cache prefetching.

For data to be prefetched from the main memory to the cache, different models have been designed. Effective implementations such as ADORE [3] and Trident [4] make use of threads to perform data cache prefetching. These systems require costly analyses and runtime code generation. The large overhead of these activities makes the dynamic cache prefetching suitable only for the code that is to be invoked multiple times.

Our strategy reduces the overhead of dynamic cache prefetching by generating *templates* at compile time, thereby avoiding costly activities such as code generation, allocation of registers and scheduling of instructions.

The main steps of runtime data cache prefetching are given below.

- (i) A compile time analysis is performed to gather information about candidate variables.
- (ii) A cost/benefit analysis is then performed that takes into account the runtime overhead that may occur for dynamic prefetching. Dynamic prefetching is performed only if the latencies of memory accesses are larger than the runtime overhead.
- (iii) If it is beneficial to perform dynamic prefetching, then the code is instrumented with prefetch instructions and the call to the runtime specializer. The code is compiled with different specialized values of offset, and one of the versions is selected as a *template*. An analysis of the versions is performed to generate the specializer code that is able to fill the *template* slots with new values during execution. The runtime specializer requires the calculation of addresses of the *template* at which the values are to be modified. In addition, the new formulae, based upon which the new values (for runtime adaptation of offsets) are generated, are also computed at compile time. These formulae take into account the (runtime) latency of the loop iterations.

The details of each of the above-mentioned steps are given in the following subsections.

3.1. Static compile-time activities

The data cache prefetching approach incorporates the selection of variables, cost/benefit analysis and generation of *templates* that are performed at (static) compile time.

3.1.1. Selection of variables

An offline profile corresponding to the latencies of memory accesses of the variables (being used in the loops) is generated. Subsequently, the profile is analyzed and the variables are selected for prefetching as given below.

Let $V = \{v_1, v_2, \dots, v_n\}$ be a set of n variables each with a sum of latencies Sv_i computed as follows:

$$Sv_i = \sum_{k=1}^{T_i} L_{k,i} \quad \forall i \in \{1, 2, \dots, n\}, \quad (1)$$

where T_i is the number of instances and $L_{k,i}$ is the latency of k th instance of the variable v_i .

The prefetch of a single cache line may bring multiple elements into the cache. The variables are therefore grouped, so that, $\forall k \in \{1, 2, \dots, T_i\}$ and $\forall j \in \{1, 2, \dots, n\}$, we have

$$V = V - \{v_j \mid L_{k,j} = 0\}. \quad (2)$$

The set of variables V having $|V| = q$ number of elements now needs to be taken into consideration to be brought into the cache.

3.1.2. Cost/benefit analysis

To estimate the effectiveness of the cache prefetching with dynamic adaptation, a cost/benefit analysis is performed. We take into account the overhead of runtime modification of the binary *template*. This is computable at compile time since the overhead related to dynamic activities including calculation of the distance formula and runtime *template* modification depends upon the fixed number of instructions to be modified.

Let R be the runtime overhead (CPU cycles) for dynamic adaptation. This adaptation requires modification of binary instructions with some activities performed at the beginning such as setting up the constructs used for modification. Similarly, cache flushing is also required after modification of instructions of the *template*. Therefore, for p instructions to be modified, we have

$$R = B + \sum_{i=1}^p M_i + F, \quad (3)$$

where M_i is the overhead of modifying a single instruction, and B and F are the overheads³ of initialization and cache flushing, respectively. Dynamic cache prefetching may be performed only when the sum of the latencies (of initial instances) is greater than the overhead that might be incurred, that is,

$$\sum_{i=1}^q L_{1,i} > R. \quad (4)$$

As there are large latencies of cache misses, especially for processors such as Itanium-II, this criterion enables one to prefetch data for a large size of code.

3.1.3. Generation of template for runtime adaptation

To adapt the prefetch distance dynamically, the offsets with respect to the base prefetch address are modified during

³All these quantities are architecture specific.

execution of the application. To accomplish this task, a *template* is generated. The *template* is a generic code that can be converted at runtime to prefetch data for different latencies of the candidate loop. It is based on a version that is compiled after providing a constant offset to the prefetch address, e.g. *lfetch(&X[i + 7])*.

Once the code is compiled with the known offset, the object code contains data dependence, which is a function of the known offset. Within this object code, different immediate constants are used for adaptation of prefetch addresses. Each immediate constant generated in the object code due to a known offset is based on an affine formula of the following form:

$$I = O * A + B, \quad (5)$$

where A and B are coefficients,⁴ and O is the offset with which the code was compiled. The instruction with the immediate constant I can be modified to adapt code with respect to any other offset. Such adaptation is required when the latencies of the operations change in the loop. The latency is computed (by using processor-specific instructions) at iteration numbers that are powers of 2, in order to keep the runtime overhead minimum. This is based on the assumption that for most of the applications, a large variance in behavior occurs during the initial time period of execution. In addition, comparison with powers of 2 may easily be implemented through shift operations in order to minimize the overhead.

The new runtime offset O_r , computed for the above-mentioned iterations, is given by the following formula:

$$O_r = \frac{\text{Avg. Cache Miss Latency}}{\text{Avg. Iteration Latency}}. \quad (6)$$

During execution, the new immediate value used for adaptation of the prefetch address is obtained by putting the value of O_r in the affine formula as given in Equation (7). We have

$$I_r = O_r * A + B, \quad (7)$$

where I_r is the new immediate constant that would replace I in the instruction (object code).

To calculate the location of each instruction that needs to be modified, and to find the values of A and B , the code is compiled with two different offsets. It results in two versions as given in Fig. 2a and b. An analysis of the two object code versions (generated with two different offsets) shows that these versions differ at corresponding locations by immediate constants.⁵ These immediate constants are a function of the offsets and based on the affine formulae. For two offsets O_1 and O_2 used to generate two versions having difference at p

```

add r15=6, r10
lfetch.nt1 [r15]
....
add r17=754, r11 ;;
lfetch.nt1 [r17]
....

```

FIGURE 3. Runtime view of the code with new prefetch distance as computed for the new latency of the loop work, with runtime offset $O_r = 6$.

locations, we have

$$I_j = O_1 * A_j + B_j \quad \forall j \in \{1, 2, \dots, p\}, \quad (8)$$

$$I_j = O_2 * A_j + B_j \quad \forall j \in \{1, 2, \dots, p\}. \quad (9)$$

Any of these versions may be used as a *template* that will be modified by the runtime specializer. The *template* and the runtime specializer, both will be linked with the application for execution. The modification of instructions of the *template* actually facilitates the runtime adaptation with new offsets based on latencies of the loop iterations. The runtime specializer will require the addresses of the locations to be modified, which are easily found at compile time by comparing the two versions. Similarly, the coefficients A_j and B_j , $\forall j \in \{1, 2, \dots, p\}$, are also found at compile time by solving the affine equations of the two versions.

3.2. Dynamic adaptation

The prefetch distance can be adapted at runtime by finding the new value of offset which is computed by taking into account the latencies of the loop iterations (Equation (6)). Subsequently, the instructions dependent on the offset are modified. The runtime specializer modifies the offset by replacing immediate values with new values.

For a new offset O_r , obtained at runtime (using Equation (6)), we need to modify all the p locations of the *template* to replace constants with runtime values. For p locations, the new immediate constants I_{rj} can be generated by using the affine formulae (given in Equations (8) and (9)), in order to adapt the code for the runtime offset O_r . Therefore, at runtime we have

$$I_{rj} = O_r * A_j + B_j \quad \forall j \in \{1, 2, \dots, p\}. \quad (10)$$

Figure 3 shows the code adapted to the new prefetch distance with the runtime offset of 6. The runtime offset is dependent upon the latency of the loop iterations, as given in Equation (6). The latency of the loop iterations is only known during execution, thereby generating accurate prefetch addresses.

4. IMPLEMENTATION ARCHITECTURE

A prototype framework has been developed that implements the steps given in Fig. 4 to perform dynamic cache prefetching with adaptation.

⁴These values are usually based on the unroll factor and data sizes as incorporated by the compilers, and can easily be found by solving affine equations.

⁵A template may only be generated when differences occur in the two versions at corresponding locations.

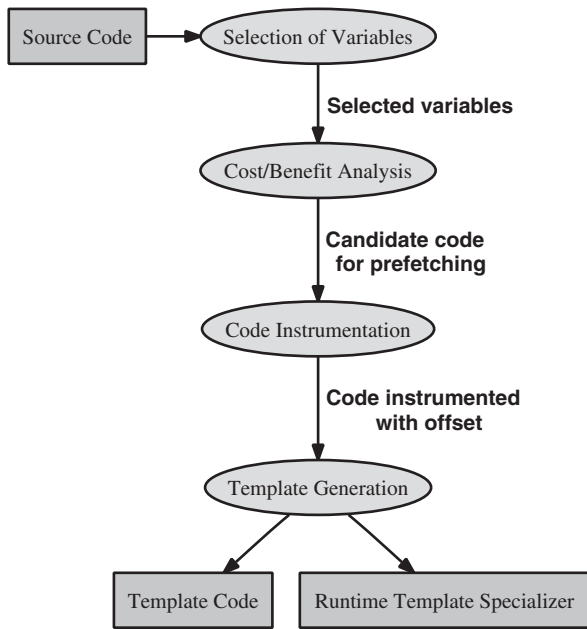


FIGURE 4. Implementation framework for performing dynamic cache prefetching.

The variables in the loops are selected by taking into account the memory access latencies as described in Section 3.1.1. A cost/benefit analysis then estimates the possible gain in performance that could be achieved by prefetching data. The approach proceeds for runtime prefetching if the estimated overhead of data prefetching is less than the performance improvement due to data prefetching.

Once the variables are selected, the source code is instrumented with prefetch macros⁶ containing two different offsets resulting in two different versions. Subsequently, the code is compiled to obtain two versions of object code as shown in Fig. 2a and b.

The two versions thus obtained differ in immediate constants at corresponding locations. The code scheduling and the optimizations incorporated in these versions are similar. The constants by which the versions differ are actually based on the offset, and have the form $offset * A + B$.

The offset in the code is replaced with the runtime value that depends upon the runtime latencies, as given in Section 3.13. These latencies are computed by instrumenting the code with hardware instructions that provide the accurate number of CPU cycles consumed for the execution of code.

Performing a comparison of the two versions exposes the instructions being used for the calculation of the prefetch distance. One of these two versions is selected as a *template* whose instructions will be modified at runtime to adapt the code for the new prefetch distance. For each instruction that differs in

⁶These macros take as arguments variable names and offsets, and are inserted outside the loop body and inside the loop to prefetch data.

```

Or = Runtime offset
void RuntimeAdaptation()
{
    //Initialization Code ...
    ModifyAdd(InstructionAddress1, Or * 1 + 0);
    ModifyAdd(InstructionAddress2, Or * 12 + 652);
    //Cache Coherence Code ...
    ...
}
  
```

FIGURE 5. Pseudo-code of the *template* specialist that adapts the offset (used in prefetching) by modifying instructions at runtime.

the two versions, the coefficients A and B are found by solving the equations. The values of A and B are subsequently used by the runtime specialist (pseudo-code shown in Fig. 5) to modify the code at runtime.

With the *ModifyAdd* macro, the code modification requires storing different values at specific locations of the binary code. For the Itanium-II processor, these values are found by using bitwise operations (depending upon the instruction location in the instruction bundle), and are subsequently stored. Since these values are stored in the cache, the modification of instructions is very efficient.

The specialist is also provided with the address of each instruction to be modified. The specialist inserts new immediate values at *add* instructions as computed by the affine formula. A runtime view of the *template* after modification of values for the new runtime offset of 6, has been shown in Fig. 3.

After inserting new immediate values, the specialist performs activities required for cache coherence on the Itanium-II architecture. All the runtime activities incur a very small overhead that is amortized by the gain in the prefetching of data.

5. EXPERIMENTAL RESULTS

The dynamic cache prefetching approach has been applied to *SPEC* benchmarks with reference inputs. The compilers *icc v 9.0* and *gcc v 4.0* are used on a system having an Itanium-II processor (with 2 GB RAM, Linux OS installed) for these experiments.

Figure 6 shows the speedup percentage obtained over *gcc* and *icc* compilers through dynamic prefetching with their prefetching options disabled.

The benchmarks *173.applu*, *179.art* and *188.ammp* are able to obtain a good speedup. The better performance is achieved due to the low overhead incurred at runtime for performing dynamic prefetching. In addition, static analysis and the selection criteria defined in our approach further minimize the overhead by applying prefetching to a small number of candidates. For the *icc* compiler, the dynamic prefetching approach achieves an average speedup of 2.51%, whereas for the *gcc*, the approach achieves an average speedup of 2.56%.

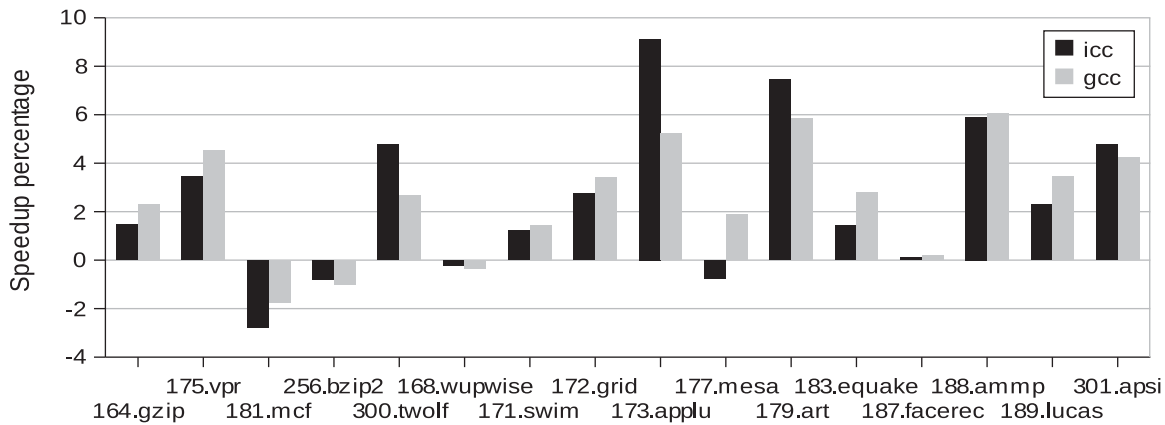


FIGURE 6. Speedup obtained due to dynamic prefetching applied to SPEC benchmarks, with compiler prefetching disabled.

The cost/benefit analysis incorporated in our approach enables one to calculate the runtime overhead at compile time. The criteria is subsequently used for the selection of variables to estimate a likely gain in performance. With prefetching enabled by the compilers, there is still (although relatively small) speedup obtained by the dynamic cache prefetching approach. For the *icc* compiler, the dynamic prefetching approach achieves an average speedup of 1.77%, whereas for the *gcc*, the approach achieves an average speedup of 1.83%.

To describe the effectiveness of the dynamic prefetching approach, a metric *cache miss reduction factor (CMRF)*, computed as

$$100 * \frac{\text{Reduction in cache misses due to dynamic prefetching}}{\text{Reduction in cache misses due to compiler prefetching}},$$

has been shown in Fig. 7. The dynamic prefetching approach has been able to achieve a significant *CMRF* value which is an improvement over (static) compile time prefetching strategies. For the *icc* compiler, the *CMRF* value reaches up to 53.65% with an average of 20.62%. Similarly, for the *gcc* compiler, the *CMRF*

value reaches up to 88% with an average of 26.47%. The large *CMRF* value shows that the cost/benefit analysis performed at compile time finds effective candidates of prefetching.

Figure 8 shows the overhead incurred due to dynamic cache prefetching for the *SPEC* benchmarks with respect to the total execution time. The overhead includes the cost of modifying the binary instructions and adaptation of prefetching performed during execution of the application. For benchmarks such as *179.art*, the overhead of dynamic adaptation is small, thereby resulting in improved performance of the application. For other benchmarks, the cost of this dynamic adaptation increases due to large variance in the execution behavior of the application.

6. RELATED WORK

Cache prefetching has been widely studied in the past to address the problem of large memory latencies, which occurs when the processor requires data from memory.

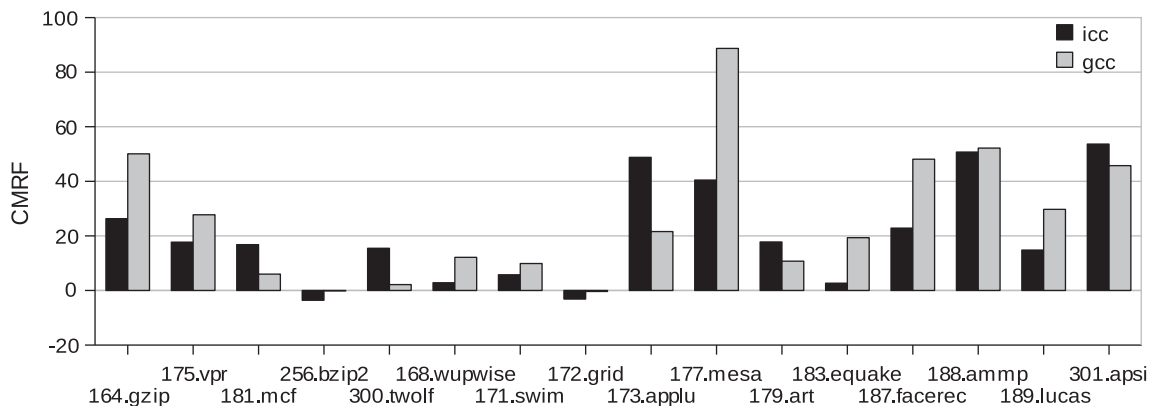


FIGURE 7. CMRF value calculated for SPEC benchmarks. The large *CMRF* value shows that the dynamic prefetching with adaptation is much better than the static prefetching.

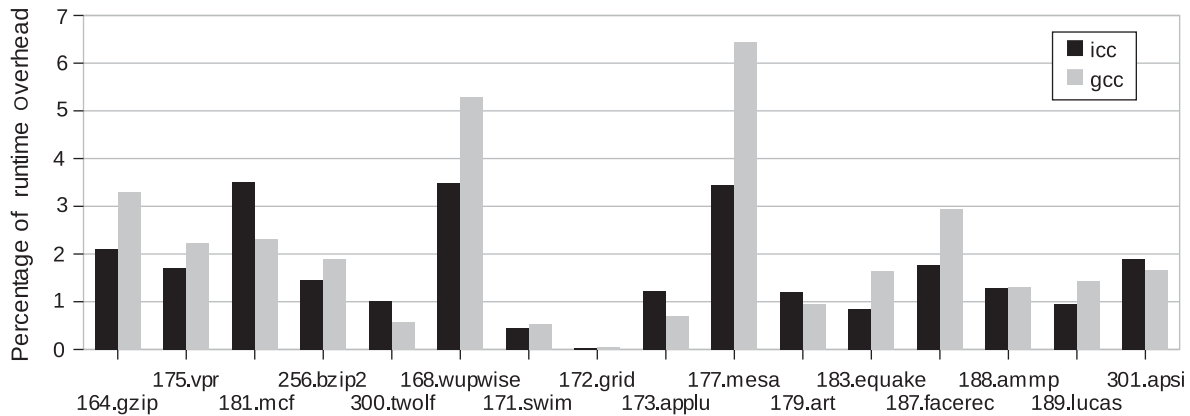


FIGURE 8. Overhead incurred (with respect to total execution time) due to dynamic prefetching applied to SPEC benchmarks.

It was observed that many applications suffer from cache misses, and so the initial work is oriented toward calculation of patterns and conditions upon which the prefetching of the cache may be based. An approach of prefetching was suggested in [6] that was able to detect the references of dense matrices which could produce cache misses. Due to cache prefetching, the performance improves up to a factor of 2. Similarly, a stride prefetching approach was given by Luk *et al.* [7]. The stride accesses are traced in order to guide the prefetching of data. Another approach of theirs performed prefetching of data for recursive data structures [8].

For linked data structures, a data flow analysis has been proposed in [5]. The analysis facilitates jump-pointer prefetching, which is implemented for the Java compiler. Similarly, another software/hardware-based jump-pointer prefetching approach is described in [9], which stores pointers at nodes at some distance in order to mitigate the latency of traversing linked data structures.

Dynamic prefetching approaches have also been implemented in the dynamic optimization systems such as ADORE [3] and Trident [4]. Both of these systems employ continuous monitoring of applications using separate threads. ADORE is a system that enables dynamic software cache prefetching for both data and instruction caches. It is able to trace the execution of an application and can optimize code based on information gathered through hardware counters. This system tracks the delinquent loads, determines the data reference pattern, calculates the stride and schedules the prefetch instructions. The trace is converted from intermediate form to binary form and the code is modified to be redirected to a new generated trace. However, for a large trace, the activities may not prove to be beneficial due to heavyweight code generation and scheduling. Trident is a system that is similar to our approach in that it adapts the prefetching to the runtime behavior of the application. The system detects the hot traces and stores information related to the delinquent load in a table. For stride-based loads, it inserts prefetches with a distance of 1, continuously observes the loads

and modifies the distance without generating another trace. The adaptation continues until there are no more delinquent loads; however, this continuous process works well only for traces having a large number of delinquent loads. The runtime activities and continuous profiling at runtime incur a large overhead, which in turn degrades the performance of the application.

For hardware-based prefetching, the data cache prefetching with precomputation has been given in [10]. Instead of predicting the prefetch addresses, the instructions producing the cache misses are precomputed. The dependence graphs are generated during execution, and are used to obtain the addresses of the load/store instructions. This approach improves the performance of the applications by 13%. It differs from our approach, where all the analyses are performed at compile time, thereby incurring minimum overhead at runtime. Similarly, another approach of speculative precomputation is introduced in [2]. For prefetching data, this approach uses idle threads to benefit from multi-threaded architectures for sequential applications, and produces an average speedup of 76%. Other prefetching strategies [11–14] improve the thread-based architectures for reducing the latency of memory access through prefetching. In contrast with these approaches, our approach of dynamic prefetching with adaptation does not require the support of multiple threads in order to prefetch data, thereby increasing the portability of its implementation to a large number of processors.

Our previous approaches of dynamic code specialization [15–17] incur a relatively small overhead for optimization, however, these approaches do not target data cache prefetching. These strategies incorporate the concept of template specialization by modifying the binary code at runtime. The decision of optimization is left to the compiler after the specialization of code. In contrast to these approaches, the cache prefetching with the dynamic adaptation approach suggested in this paper is not dependent upon the compiler.

Other dynamic optimization systems such as Fabius [18], Tempo [19, 20] or other approaches given in [21–24] exploit

runtime information to optimize code during execution of the application. Since dynamic code generation requires analyses at runtime, these approaches incur a very large overhead. This overhead may only be amortized when the optimized code is invoked multiple times.

7. CONCLUSION

This paper presents an approach that exploits runtime information for accurate prefetching of data. Dynamic cache prefetching is adapted during execution to work in accordance with varying latencies of loop iterations.

For variables selected after analysis, the code is instrumented with prefetch instructions. The prefetch address contains offsets that are adapted at runtime by a specializer. The code is then compiled with known offsets to generate a *template*. An analysis is subsequently performed to search for instructions of the *template* that must be modified to adapt prefetches to new latencies. The runtime specialization incurs a very small overhead since a few instructions are modified and all analyses are performed at (static) compile time.

The accurate and timely prefetching of data results in reduction of cache misses and thereby produces significant improvement in the performance. For *SPEC* benchmarks, the approach enables one to produce an overall average speedup of 2.17%. In addition, the dynamic prefetching approach reduces cache misses by an average of 23.54% in comparison with static cache prefetching performed by compilers.

As future work, the approach is being extended to incorporate some effective transformations in the runtime specialization system. This would result in better performance by exploiting more runtime information obtained during execution of the application.

REFERENCES

- [1] Lu, J., Chen, H., Fu, R., Hsu, W.-C., Othmer, B., Yew, P.-C. and Chen, D.-Y. (2003) The Performance of Runtime Data Cache Prefetching in a Dynamic Optimization System. *MICRO 36: Proc. 36th Annual IEEE/ACM Int. Symp. Microarchitecture*, Washington, DC, USA, p. 180. IEEE Computer Society.
- [2] Collins, J.D., Wang, H., Tullsen, D.M., Hughes, C., Lee, Y.-F., Lavery, D. and Shen, J.P. (2001) Speculative Precomputation: Long-Range Prefetching of Delinquent Loads. *ISCA'01: Proc. 28th Annual Int. Symp. Computer Architecture*, New York, NY, USA, pp. 14–25. ACM.
- [3] Lu, J., Chen, H., Yew, P.-C. and Hsu, W.-C. (2004) Design and implementation of a lightweight dynamic optimization system. *J. Instr.-Level Parallelism*, **6**, 103–126.
- [4] Zhang, W., Calder, B. and Tullsen, D. (2005) An Event-Driven Multithreaded Dynamic Optimization Framework. *Int. Conf. Parallel Architectures and Compilation Techniques (PACT)*, St. Louis, MO, USA, September.
- [5] Cahoon, B. and McKinley, K.S. (2001) Data Flow Analysis for Software Prefetching Linked Data Structures in Java. *Int. Conf. Parallel Architectures and Compilation Techniques*, Barcelona, Spain, pp. 280–291.
- [6] Mowry, T.C., Lam, M.S., and Gupta, A. (1992) Design and Evaluation of a Compiler Algorithm for Prefetching. *Proc. Fifth Int. Conf. Architectural Support for Programming Languages and Operating Systems*, Boston, MA, USA, pp. 62–73.
- [7] Luk, C.-K., Muth, R., Patil, H., Weiss, R., Lowney, P.G. and Cohn, R. (2002) Profile-Guided Post-Link Stride Prefetching. *ICS'02: Proc. 16th Int. Conf. Supercomputing*, New York, NY, USA, pp. 167–178. ACM.
- [8] Luk, C.-K. and Mowry, T.C. (1996) Compiler-Based Prefetching for Recursive Data Structures. *Proc. Seventh Int. Conf. Architectural Support for Programming Languages and Operating Systems*, Cambridge, MA, USA, pp. 222–233.
- [9] Roth, A. and Sohi, G.S. (1999) Effective jump-pointer prefetching for linked data structures. *SIGARCH Comput. Archit. News*, **27**, 111–121.
- [10] Jignesh, M.A., Patel, J.M. and Davidson, E.S. (2001) Data Prefetching by Dependence Graph Precomputation. *Proc. 28th Annual Int. Symp. Computer Architecture*, Goteborg, Sweden, pp. 52–61. ACM.
- [11] Luk, C.-K. (2001) Tolerating Memory Latency Through Software-Controlled Pre-execution in Simultaneous Multithreading Processors. *ISCA'01: Proc. 28th Annual Int. Symp. Computer Architecture*, New York, NY, USA, pp. 40–51. ACM.
- [12] Collins, J., Sair, S., Calder, B. and Tullsen, D.M. (2002) Pointer Cache Assisted. *35th Int. Symp. Microarchitecture*, Istanbul, Turkey, pp. 62–73. IEEE Computer Society.
- [13] Roth, A. and Sohi, G.S. (2001) Speculative Data-Driven Multithreading. *HPCA'01: Proc. 7th Int. Symp. High-Performance Computer Architecture*, Washington, DC, USA, p. 37. IEEE Computer Society.
- [14] Zilles, C. and Sohi, G. (2001) Execution-Based Prediction Using Speculative Slices. *ISCA'01: Proc. 28th Annual Int. Symp. Computer Architecture*, New York, NY, USA, pp. 2–13. ACM.
- [15] Khan, M.A. (2009) Feedback-directed specialization of code. *Comput. Lang., Syst. Struct.*, **35**, 1–10.
- [16] Khan, M.A., Charles, H.-P. and Barthou, D. (2008) An Effective Automated Approach to Specialization of Code. *Proc. 20th Int. Workshop on Languages and Compilers for Parallel Computing*, 11–14 October 2007, Urbana, IL, USA.
- [17] Khan, M.A., Charles, H.-P. and Barthou, D. (2007) Reducing Code Size Explosion Through Low-Overhead Specialization. *Proc. 11th Annual Workshop on the Interaction between Compilers and Computer Architecture*, Phoenix, February.
- [18] Leone, M. and Lee, P. (1998) Dynamic specialization in the Fabius system. *ACM Comput. Surv.*, **30**, 23–23.
- [19] Noel, F., Hornof, L., Consel, C. and Lawall, J.L. (1998) Automatic, Template-Based Run-Time Specialization: Implementation and Experimental Study. *Int. Conf. Computer Languages (ICCL'98)*, Chicago, USA, February.
- [20] Consel, C., Hornof, L., Noel, F., Noyé, J. and Volanschi, N. (1996) A Uniform Approach for Compile-Time and Run-Time Specialization. *Partial Evaluation. Int. Seminar*, Dagstuhl Castle, Germany, pp. 54–72. Springer, Berlin, Germany.

- [21] Leone, M. and Lee, P. (1996) A Declarative Approach to Run-Time Code Generation. *Workshop on Compiler Support for System Software (WCSSS)*, Arizona, USA, February.
- [22] Grant, B., Mock, M., Philipose, M., Chambers, C. and Eggers, S.J. (1997) Annotation-Directed Run-Time Specialization in C. *Proc. ACM SIGPLAN Symp. Partial Evaluation and Semantics-Based Program Manipulation (PEPM'97)*, Amsterdam, Netherlands, June, pp. 163–178. ACM.
- [23] Engler, D. (1996) Vcode: A Retargetable, Extensible, Very Fast Dynamic Code Generation System. *Proc. ACM SIGPLAN 96 Conf. Programming Language Design and Implementation*, Pennsylvania, USA, pp. 160–170. ACM.
- [24] Leone, M. and Dybvig, R.K. (1997) Dynamo: A Staged Compiler Architecture for Dynamic Program Optimization. Technical Report, Indiana University.